

SLE-3: Architectural Design (Full C4 Model)

Course: 02AML204 – Introduction to Artificial Intelligence

PRN: 25UAM112	Name: Rajdeep Mahadev Kamble
Division: B	Date: 04 October 2026
GitHub: https://github.com/RajDev20	System: 8-Puzzle Solver (from SLE-2)

1. System Title & Short Description

8-Puzzle Solver with BFS vs DFS Profiling. This is the same system I profiled in SLE-2. The 8-puzzle is a 3x3 sliding-tile puzzle where the blank tile moves up, down, left or right until the tiles match the goal arrangement. The system takes a solvable start state, solves it using two uninformed search algorithms (Breadth-First Search and Depth-First Search), and records time, nodes expanded and solution length for each. It solves the problem of choosing a suitable search strategy with measured evidence: in SLE-2, BFS found the optimal 13-move path while DFS needed a 32,669-move path. Since no heuristic is used, there is no A* module.

2. Context Diagram (Level 1)

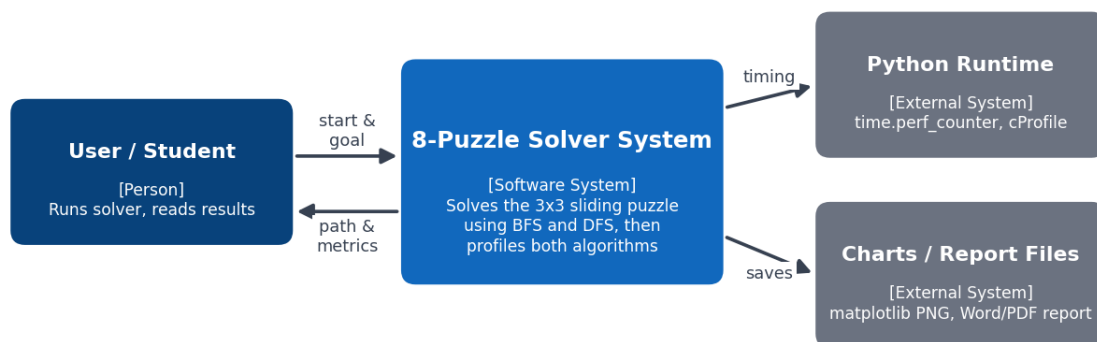


Figure 1 - Level 1: System Context

The **User / Student** gives a start and goal state to the **8-Puzzle Solver System** and receives the solution path and performance metrics. The system depends on two external things: the **Python Runtime** (time.perf_counter and cProfile for measurement) and **Chart / Report files** (matplotlib PNG charts and the Word/PDF report) where results are saved.

3. Container Diagram (Level 2)

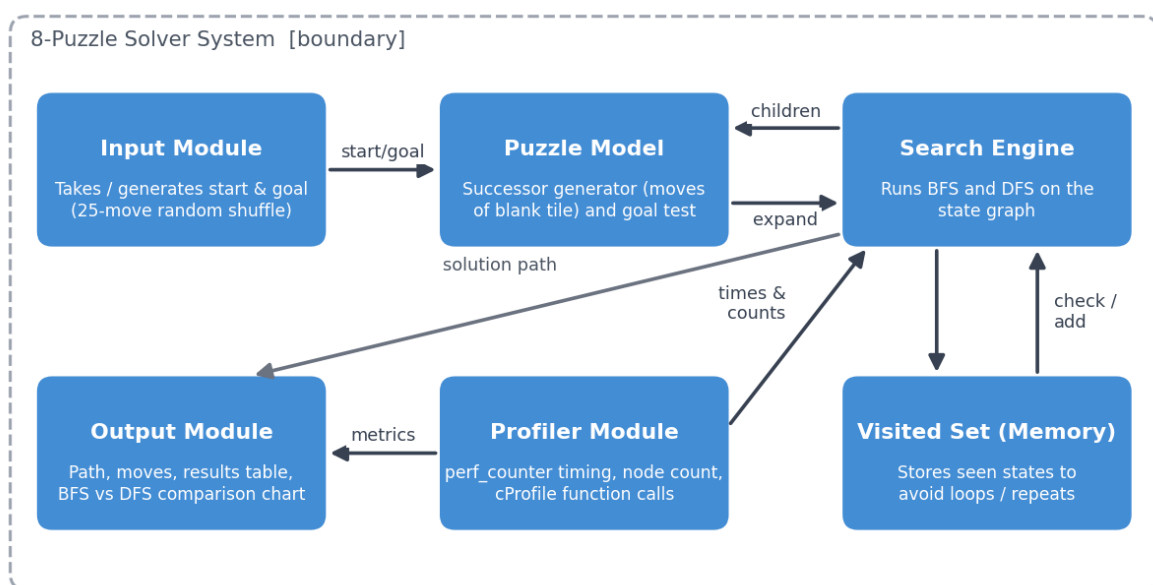


Figure 2 - Level 2: Containers (6 boxes)

Explanation of each container:

- **Input Module:** creates or accepts the start and goal state; the start is a 25-move random shuffle of the goal, so it is always solvable.
- **Puzzle Model:** defines the problem: generates legal successor states (blank-tile moves) and tests whether a state is the goal.
- **Search Engine:** runs BFS and DFS over the state graph and returns the solution path.
- **Visited Set (Memory):** stores states already seen so the search never loops or re-expands the same state.
- **Profiler Module:** times each run (5 runs, perf_counter), counts expanded nodes and collects cProfile function-call data.
- **Output Module:** shows the path and number of moves, the results table and the BFS vs DFS comparison chart.

4. Component Diagram (Level 3)

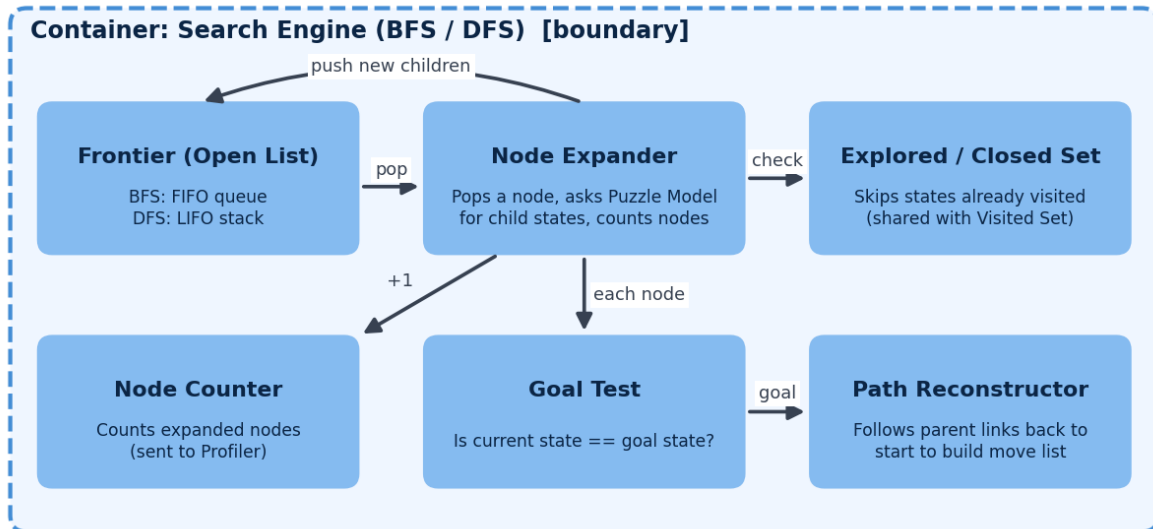


Figure 3 - Level 3: Components inside the Search Engine

The Search Engine is the most important container, so only it is expanded. The **Frontier** is a FIFO queue for BFS and a LIFO stack for DFS; this is the only structural difference between the two algorithms. The **Node Expander** pops a node, asks the Puzzle Model for its children, and checks the **Explored Set** before pushing new children back to the Frontier. The **Goal Test** runs on each expanded node; when it succeeds, the **Path Reconstructor** follows parent links back to the start. The **Node Counter** provides the nodes-expanded value used by the Profiler.

5. Code Level Overview (Level 4)

Main classes / functions (names and one-line responsibility only):

- **class Node** : stores a puzzle state, its parent node and the move that led to it.
- **class PuzzleProblem** : holds the start and goal state; provides the successor and goal-test operations.
- **def get_successors(state)** : returns all legal states reachable by moving the blank tile.
- **def is_goal(state)** : returns True when the state equals the goal arrangement.
- **def bfs(problem)** : uses a queue (FIFO); returns the shortest path and nodes expanded.
- **def dfs(problem)** : uses a stack (LIFO) with a visited set; returns a path and nodes expanded.
- **def reconstruct_path(node)** : walks parent links from the goal node back to the start.
- **def profile_run(algo, problem)** : times the algorithm with perf_counter and runs cProfile.

6. Design Decisions

I separated the **Puzzle Model** from the **Search Engine** so that BFS and DFS use exactly the same move rules and goal test; only the Frontier type changes. This makes the SLE-2 comparison fair and lets me add another algorithm (such as A*) later without touching the puzzle logic. A separate **Visited Set** prevents infinite loops, since the 8-puzzle has 181,440 reachable states. The **Profiler** is kept outside the search code so measurement does not mix with algorithm logic. I kept the design to 6 containers to stay readable.

7. AI Contribution Note

- **AI tools used:** Claude (Anthropic).
- **What AI helped with:** suggesting the container and component breakdown, drawing the three diagrams, and drafting the explanations in this report.
- **What I did myself:** chose the 8-Puzzle system from my SLE-2 work, reviewed every diagram against my own code, checked that the class and function names match my implementation, and made sure I can explain each design choice.

8. Conclusion

This SLE showed me that architecture can be explained at four zoom levels: Context shows who uses the system, Container shows its main building blocks, Component shows the inside of one block, and Code shows the key classes and functions. Drawing the Search Engine in detail made it clear that BFS and DFS differ only in their Frontier (queue vs stack), which explains the large difference I measured in SLE-2. A clear separation of responsibilities makes a system easier to explain, test and extend.